

Se proteger de l'injection SQL

Exercice 3 - La requete preparee, expliquee simplement

KHEN - Lab SOC | 22 juillet 2026

Rouge = danger / attaque

Vert = protection

Gris = a retenir

1. C'est quoi une injection SQL ? (en simple)

Une application pose sans arrêt des questions a sa base de donnees, par exemple : « *donne-moi la personne dont l'email est X* ». Jusque-la, tout est normal.

Le probleme arrive quand le code **colle directement** ce que tape le visiteur dans la question, sans le controler. Si le visiteur ne met pas un email, mais un **bout de commande**, ce bout se melange a la question et en **change le sens**.

L'idée cle : La base ne fait pas la difference entre la **question** (ecrite par le code) et la **valeur** (tapee par le visiteur). Resultat : le visiteur peut donner des **ordres** a la base. C'est ca, l'injection SQL : une simple donnee se transforme en commande.

2. La demonstration (teste dans le navigateur)

Sur la page **vulnerable** (login.php), dans le champ Email, j'ai tape :

```
' OR '1'='1' --
```

Le code a alors construit litteralement cette phrase (ma saisie est venue se coller a la place de l'email) :

```
SELECT id, nom, prenom, email FROM users
WHERE email = '' OR '1'='1' -- ' AND mdp = ''
```

Deux details font tout basculer :

- Le -- est un **commentaire** : tout ce qui suit est **ignore** par la base. Donc la partie ' AND mdp = '' est rayee : le mot de passe ne compte plus.
- Il reste email = '' OR '1'='1'. Or '1'='1' est **toujours vrai**. La condition est donc vraie pour tout le monde.

Vulnerable : Resultat sur la page vulnerable : « **Connecte ! Bienvenue...** » - entre **sans mot de passe valide**. L'attaque a reussi.

Sur la page **protegee** (login_secure.php), j'ai tape **exactement le meme texte** :

Protegee : Resultat sur la page protegee : « **Identifiant ou mot de passe invalide** ». Meme attaque, mais ici la base l'a traitee comme du **texte simple** : l'attaque a echoue.

3. La correction : la requete preparee

Avant (dangereux) - la saisie est collee direct dans la question :

```
// login.php (VULNERABLE)
$email = $_REQUEST['email'];
$sql = "SELECT id, nom, prenom, email FROM users
      WHERE email = '$email' AND mdp = '$mdp'";
$bdd->query($sql);
```

Après (sur) - la question et la valeur sont envoyées **separement :**

```
// login_secure.php (PROTEGE - requete preparee)
$stmt = $bdd->prepare(
    "SELECT id, nom, prenom, email FROM users WHERE email = ? AND mdp = ?"
);
$stmt->execute([$email, $mdp]);
```

Pourquoi ça protège : En simple : on donne d'abord à la base la phrase avec des **trous** (les ?), puis les valeurs à part. La base est prévenue que ces trous sont **uniquement des valeurs à chercher, jamais des ordres**. Même si l'attaquant y met sa phrase piégée, elle est cherchée comme du texte : l'injection ne peut plus marcher.

4. Les 3 questions de l'exercice

Pourquoi la première requête est-elle dangereuse ?

Parce qu'elle colle directement ce que tape le visiteur dans la question envoyée à la base, sans le contrôler. Si le visiteur met un bout de commande au lieu d'un email, ce bout se mélange à la question et en change le sens : le visiteur peut donner des ordres à la base.

Que pourrait faire un attaquant s'il trouve cette faille ?

- Se **connecter sans mot de passe** (contourner l'authentification).
- **Extraire toute la base** (emails, mots de passe, données personnelles).
- **Lire, modifier ou supprimer** des données qui ne le regardent pas.
- Dans les cas graves, aller jusqu'à **prendre la main sur le serveur**.

Comment la requête préparée protège-t-elle la base ?

Elle envoie la phrase et la valeur séparément : d'abord la structure figée avec un trou (. . . WHERE email = ?), puis la valeur à part. La base sait que ce trou est uniquement une valeur à chercher, jamais un ordre. Même avec ' OR '1'='1' -- dedans, elle cherche un email qui vaut littéralement ce texte, n'en trouve aucun, et ne renvoie rien. L'injection est neutralisée.

5. A retenir (bonnes pratiques)

- **Ne jamais coller** la saisie d'un utilisateur directement dans une requête SQL.
- **Toujours utiliser des requêtes préparées** : PDO `execute([ . . . ])` ou `mysqli bind_param()`.
- Ne pas afficher les erreurs SQL brutes à l'écran (ça aide l'attaquant).
- Bonus : stocker les mots de passe **haches** (`password_hash()`), jamais en clair.

Exercice 3 valide : injection comprise, démontée (avant / après), corrigée par requête préparée. Lien avec l'exercice 2B (attaque SQLMap + détection Wazuh) : la requête préparée est la remédiation de cette faille.